

Reinforcement Learning Summative Assignment Report

Student Name: Nziza Aime Pacifique

Video Recording: <https://drive.google.com/file/d/1ZsFxY9nqWZWnrMkExikMvVPvDAjsUNLA/view?usp=sharing>

GitHub Repository: <https://github.com/NzizaPacifique250/Education-Based-Reinforcement-Learning>

Environment: SchoolCheckIn-v0, a custom Gymnasium environment rendered in 3D with PyBullet

Project Overview

This project trains an agent to run the morning attendance routine at a school that uses biometric check-in. A pupil arrives at the gate and has to register attendance before the late bell by reaching a fingerprint reader and scanning in successfully. The reader only accepts a clean finger, hands can only be washed at a sanitizer stand on the far side of the courtyard, one of the two readers is unreliable and is sometimes out of service, and both readers queue during the morning rush. A good policy therefore has to plan a route in stages, keep track of which reader is still usable, and decide when to keep trying rather than give up. I compare a value based method (DQN) against three policy based methods (REINFORCE, PPO and A2C) on exactly the same environment, with ten hyperparameter configurations each.



Figure 1. The environment rendered in PyBullet. The classroom block sits along the north side with the main entrance and reader A, the east side gate carries reader B, and the courtyard holds the sanitizer stand, the reception booth, a hedge planter and a run of queue barriers.

Environment Description

Agent

The agent is a single pupil arriving at the school gate. It is drawn as a uniformed figure with a backpack, and the hand it will present to the reader is tinted by how clean it is, so you can see the hygiene state directly in the simulation. The pupil can walk in four directions on a half metre grid, use the sanitizer stand, present a finger at either reader, stand and wait for a queue to clear, call a staff member to reset a reader that has locked it out, and sign the paper register at reception once it has grounds to do so. It cannot walk through the hedge planter or the queue barriers, and bumping into them makes its hands dirty again, so careless movement has a cost later on. Its position, its hand cleanliness and the number of rejections it has collected are all it really knows. Whether reader B is broken is hidden until it gets close enough to read the display, so part of the task is finding out.

Action Space

The action space is discrete with nine actions, written as `Discrete(9)`. I kept it discrete so that DQN, REINFORCE, PPO and A2C can all be trained on exactly the same environment and compared fairly. Every action maps onto something a real pupil does at a school gate.

Id	Action	Meaning in the real world
0	move_north	Walk one step (0.5 m) towards the building.
1	move_south	Walk one step away from the building.
2	move_west	Walk one step towards the gate side.
3	move_east	Walk one step towards the reader side.
4	sanitize_hands	Take a dose of sanitizer. Only works at the stand.
5	scan_fingerprint	Present a finger to a reader. Only works at a reader.

Id	Action	Meaning in the real world
6	wait_in_queue	Stand still and let the queue ahead clear.
7	request_assistance	Call a staff member to recalibrate a reader. Takes 4 steps.
8	check_in_manually	Sign the register at reception. Only after a lockout.

Movement of 0.5 m per step matches an unhurried walking pace sampled at roughly one step per second. The three station actions only have an effect at the matching station, which is what forces the agent to plan a route instead of acting from anywhere in the courtyard.

Observation Space

The observation is a 29 element `Box` vector, every element scaled into `[-1, 1]` and stored as `float32`. The agent sees what a real deployment could actually measure, not the full internal state. In particular it does not see the true reliability of either reader, and it does not see whether reader B is broken until it is close enough to read the display. The table below lists each group of features and the equipment that would supply it on a real site.

Observation	Description	Source (sensor, camera, API, dataset)	Encoding and data type	Range
Pupil position (x, y)	Where the pupil is standing in the courtyard.	Overhead fisheye camera with a person tracker, or a UWB tag in the student ID card	float32, 2 values, scaled	[-1, 1]
Offset and range to reader A	Vector and straight line distance to the main door reader.	Same tracker, combined with the surveyed coordinates of the kiosk from the site plan	float32, 3 values	[-1, 1]
Offset and range to reader B	Vector and distance to the east side gate reader.	As above	float32, 3 values	[-1, 1]
Offset and range to sanitizer	Vector and distance to the hand sanitizer stand.	As above	float32, 3 values	[-1, 1]
Offset and range to reception	Vector and distance to the reception booth.	As above	float32, 3 values	[-1, 1]
At station flags (4)	Whether the pupil is within 0.9 m of each of the four stations.	Bluetooth Low Energy proximity beacon at each station, or a geofence over the tracker feed	float32, 4 values, 0 or 1	{0, 1}
Hand cleanliness	Estimated cleanliness of the finger that will be presented.	Dose counter on the sanitizer dispenser plus elapsed time, cross checked against the reader's own NFIQ image quality score	float32	[0, 1]
Rejections at A and B	Failed scans recorded so far at each reader.	Attendance and access control REST API, per device event log	float32, 2 values, scaled by 3	[0, 1]
Lockout flags A and B	Whether a reader has locked this pupil out.	Same access control API, device state field	float32, 2 values, 0 or 1	{0, 1}
Reader B health belief	Whether the side reader is out of service. Reads 0 until the pupil is close enough to see the display.	Device heartbeat endpoint, legible on the kiosk screen within roughly 2.5 m	float32	{-1, 0, 1}
Queue length at A and B	How many people are waiting at each reader.	Overhead camera people counter, or the queue display board feed	float32, 2 values, scaled by 3	[0, 1]
Time remaining	Fraction of the episode budget still left.	System clock	float32	[0, 1]
Time to the bell	Fraction of time left before the late bell. Goes negative once late.	School timetable API plus system clock	float32	[-1, 1]
Manual sign-in available	Whether reception will accept a signature yet.	Access control API, lockout status	float32, 0 or 1	{0, 1}

Two things are deliberately withheld. The per reader acceptance probability is never observable, so the agent has to treat rejection as noise rather than as a signal it can read off directly. The health of reader B is masked to zero beyond about 2.5 m, which makes the walk towards it an information gathering action as well as a movement.

Reward Structure

The reward has three parts: a small running cost, a shaping term that measures progress along the route, and event rewards for what the agent does at the stations. The step reward is

```

r_t = -c_step + w_d * ( PHI(s_t+1) - PHI(s_t) ) + r_event

c_step = 0.05      per step cost, so dawdling is never free
w_d     = 1.5      weight on route progress

```

The shaping potential PHI is the negative of the remaining route length. If no reader is still usable it measures the walk to reception. Otherwise it blends the direct walk to the nearest usable reader with the detour through the sanitizer, weighted by how dirty the hands currently are:

```

PHI(s) = -[ delta * ( d(p, H) + min_i d(H, S_i) ) + (1 - delta) * min_i d(p, S_i) ]

```

```
delta = clip( (0.7 - h) / 0.7, 0, 1 )      h = hand cleanliness
p = pupil position, H = sanitizer, S_i = usable readers
```

Writing the shaping this way matters more than it looks. Because PHI depends only on the state, every closed loop in state space sums to zero, so no repeating behaviour can farm it. An earlier version of the reward simply paid the agent for getting closer to whichever sub goal was active at the time. Since that sub goal flipped between the sanitizer and the reader as the hands got dirty, walking back and forth paid on both legs, and PPO learned to shuttle between the two forever. It scored about +40 per episode while checking in on 0% of episodes. The regression test `test_shaping_cannot_be_farmed_by_shuttling` now keeps that closed.

The event rewards are as follows.

```
Movement
  blocked by the planter or barriers  -0.25  and hand cleanliness h <- h - 0.20
  free step                          h <- h - 0.012

Sanitizer
  dose taken at the stand (h < 0.95)  0      h <- min(h + 0.45, 1.0)
  dose taken with clean hands         -0.15
  used away from the stand            -0.20

Fingerprint reader
  used away from a reader             -0.20
  reader locked out or out of service -0.60
  reader busy with someone else       -0.15
  accepted, probability p              +20.0  and the episode ends
    p = clip( (h - 0.45) / 0.45, 0.02, 0.97 ) * rho_i
  rejected                           -0.50  h <- h - 0.12, rejections_i <- +1
  third rejection at that reader       -2.00  that reader locks out

Other
  wait in queue                      -0.02
  request assistance at a reader      -0.30  staff clear that reader after 4 steps
  manual sign-in at reception         +5.00  only after a lockout, else -0.20

Terminal adjustments
  checked in before the bell (t <= 80) +5.0 * (80 - t) / 80
  checked in after the bell           -4.00
  every route exhausted (stranded)    -8.00
  never checked in by step 150       -6.00
```

The gap between +20 for a biometric check-in and +5 for a signature is what keeps the agent honest, and reception refuses to sign anyone that no reader has actually rejected. Without that precondition the office is a nine step walk from the gate for a guaranteed reward, and PPO simply took it: an earlier run converged on signing the register in 9 steps on 100% of episodes and never learned to use a reader at all.

Start State and Termination

Each episode starts with the pupil at the gate. Hand cleanliness is drawn uniformly from [0.15, 0.60], reader A reliability from [0.80, 1.00], reader B reliability from [0.45, 0.85], and reader B is out of service on 25% of episodes. An episode ends when the pupil scans in successfully, signs the register at reception, becomes stranded because every route is gone and reception is out of reach in the time left, or runs out of time at step 150. The bell rings at step 80 and splits punctual arrivals from late ones.

System Analysis And Design

Deep Q-Network (DQN)

DQN comes from Stable Baselines3 with the `MlpPolicy` head. The network is a fully connected value network that maps the 29 element observation to nine action values, with the hidden layer sizes varied across the sweep from a single layer of 64 units up to two layers of 256. The winning configuration, `dqn07`, uses [128, 64]. The implementation keeps the two standard stabilisers. A replay buffer holds past transitions and is sampled uniformly, which breaks the correlation between consecutive steps in a long walk across the courtyard. A target network supplies the bootstrap value and is refreshed every 500 steps in the best run, which stops the regression target moving with every gradient step. Exploration is epsilon greedy, annealed from 1.0 down to 0.05 over the fraction of training given by `exploration_fraction`. Learning starts after 1000 steps of pure exploration and a gradient step is taken every 4 environment steps.

The design question here is that the task needs a long correct sequence, roughly 35 steps, before any reward for checking in arrives. The shaped route reward carries most of the learning signal, and the replay buffer lets DQN reuse the rare successful endings many times instead of once.

Policy Gradient Methods (REINFORCE, PPO and A2C)

REINFORCE is written from scratch in PyTorch, since Stable Baselines3 does not provide it. The policy network has two hidden Tanh layers, with width varied across the sweep from 64 to 256 units, and a softmax over the nine actions through a Categorical distribution. Training is pure Monte Carlo: it plays four complete episodes, computes discounted returns to go for every visited state, and takes one gradient step on the batch. The `baseline` column in the sweep switches on a learned state value network trained by regression on the same returns, which is the REINFORCE with baseline formulation from Sutton and Barto section 13.4. Advantages are normalised before the update and an entropy bonus is available through `ent_coef`.

The baseline turned out to matter a great deal. With plain normalised returns the gradient variance over a 150 step episode with nine actions was large enough that the agent never found the route at all, sitting at a 0% check-in rate even after 400k steps. Subtracting a learned value estimate leaves the gradient unbiased but much quieter, and the method then reaches the same final performance as PPO.

PPO and **A2C** both come from Stable Baselines3 with the default `MlpPolicy`, an actor critic pair of two 64 unit hidden layers. PPO collects a rollout of `n_steps` transitions, computes advantages with generalised advantage estimation, and then takes several epochs of minibatch updates under a clipped objective, where `clip_range` bounds how far the policy may move in one update. A2C is the synchronous, single update version of the same idea, so it updates far more often on much shorter rollouts, from 5 to 64 steps in this sweep. Both expose an entropy coefficient, and as the results show, that single knob decides whether A2C learns anything at all here.

Implementation

Every algorithm was run with ten different hyperparameter configurations on the same environment. Mean return and check-in rate are measured over 30 evaluation episodes with a seed range disjoint from training, so the numbers reflect generalisation and not memorised starts. The winning row of each table is highlighted. DQN, PPO and A2C were each trained for 250k steps; REINFORCE needed 900k, for reasons discussed below.

DQN

Run	Learning rate	Gamma	Replay buffer	Batch	Exploration fraction	Target update	Hidden layers	Mean reward	Check-in rate	Steps
dqn01	0.001	0.99	50000	64	0.2	500	[64, 64]	+8.47	0.47	136.3
dqn02	0.0005	0.99	100000	128	0.3	1000	[128, 128]	+20.13	0.70	68.9
dqn03	0.0001	0.95	50000	64	0.1	500	[64, 64]	+33.61	0.90	45.6
dqn04	0.001	0.9	20000	32	0.4	250	[64]	+34.57	0.93	41.9
dqn05	0.0003	0.99	200000	256	0.25	2000	[256, 256]	+26.97	0.87	68.5
dqn06	0.0005	0.999	100000	128	0.15	1000	[128, 128]	+20.97	0.70	70.0
dqn07	0.001	0.98	50000	64	0.5	500	[128, 64]	+34.65	0.97	42.7
dqn08	0.0007	0.97	75000	128	0.2	750	[128, 128]	+34.22	0.93	42.7
dqn09	0.0002	0.99	150000	256	0.3	1500	[256, 128]	+33.34	0.93	43.6
dqn10	0.001	0.99	100000	64	0.1	1000	[64, 64]	+32.31	0.93	47.5

Learning rate and the discount factor dominate. The two runs at gamma 0.999 and 0.99 with a small learning rate stay well behind, while the three best runs all sit at gamma between 0.90 and 0.98. A shorter effective horizon suits this task because the shaped route reward already carries the long term information, so a very long horizon mostly adds variance. The clearest single lesson is in dqn01 against dqn10: same learning rate, same gamma, same hidden layers, and yet 0.47 against 0.93 check-in rate. They differ only in exploration fraction, 0.20 against 0.10, and target update interval. The longer exploration schedule left dqn01 still acting semi randomly late in training, and its episodes average 136 steps against 47, so it was reaching the reader after the bell rather than not at all.

REINFORCE

Run	Learning rate	Gamma	Hidden width	Entropy coef	Value baseline	Mean reward	Check-in rate	Late rate	Steps
reinforce01	0.001	0.99	64	0	True	-2.86	0.00	0.00	150.0
reinforce02	0.0005	0.99	128	0.01	True	+36.65	1.00	0.00	36.6
reinforce03	0.001	0.95	64	0	False	+27.46	0.87	0.00	47.9
reinforce04	0.0003	0.99	128	0.02	True	+29.55	0.93	0.03	45.4
reinforce05	0.001	0.999	256	0.01	True	-4.67	0.00	0.00	150.0
reinforce06	0.0007	0.98	64	0.05	True	+30.05	0.93	0.00	40.9
reinforce07	0.0002	0.99	128	0	False	-9.02	0.00	0.00	150.0
reinforce08	0.001	0.97	64	0.01	True	+36.68	1.00	0.00	36.5

Run	Learning rate	Gamma	Hidden width	Entropy coef	Value baseline	Mean reward	Check-in rate	Late rate	Steps
reinforce09	0.0005	0.99	256	0	True	+0.60	0.00	0.00	150.0
reinforce10	0.001	0.99	128	0.03	False	+37.23	1.00	0.00	35.8

Entropy is the deciding column. Of the four runs with no entropy bonus, three fail completely and never check in. The three runs that reach a 1.00 check-in rate all carry an entropy coefficient between 0.01 and 0.03. The one exception, `reinforce03`, gets to 0.87 with no entropy bonus but compensates with a shorter horizon at gamma 0.95. Gamma 0.999 in `reinforce05` fails outright, which matches what DQN showed. Every failing run has a mean episode length of exactly 150, meaning it never terminates and simply wanders until the clock runs out.

PPO

Run	Learning rate	Gamma	Rollout steps	Batch	Clip range	Entropy coef	GAE lambda	Mean reward	Check-in rate	Steps
ppo01	0.0003	0.99	1024	64	0.2	0	0.95	+35.60	1.00	41.8
ppo02	0.001	0.99	2048	64	0.2	0.01	0.95	+36.69	1.00	37.9
ppo03	0.0003	0.95	1024	128	0.1	0	0.9	+33.34	0.90	45.8
ppo04	0.0005	0.99	512	64	0.3	0.02	0.95	+31.48	0.93	48.0
ppo05	0.0001	0.999	2048	256	0.2	0.01	0.98	-5.96	0.33	120.7
ppo06	0.0003	0.98	1024	64	0.2	0.05	0.95	+36.14	1.00	40.9
ppo07	0.0007	0.99	2048	128	0.15	0	0.92	+21.85	0.77	92.4
ppo08	0.0002	0.97	512	64	0.2	0.01	0.95	+31.63	0.93	42.4
ppo09	0.0003	0.99	4096	256	0.25	0	0.95	+6.10	0.40	125.1
ppo10	0.0005	0.99	1024	64	0.2	0.03	0.9	+36.97	1.00	38.9

PPO is the most forgiving of the four. Seven of ten configurations pass a 0.90 check-in rate, and the failures are informative rather than random. `ppo05` pairs the smallest learning rate in the sweep with gamma 0.999 and collapses to 0.33. `ppo09` uses a 4096 step rollout, which at a 250k budget leaves only about 61 policy updates in total, and it reaches 0.40 with a 0.27 late rate. Both failures are about not making enough progress in the budget rather than about instability, which is what the clipped objective is there to prevent.

A2C

Run	Learning rate	Gamma	Rollout steps	Entropy coef	Value coef	GAE lambda	Mean reward	Check-in rate	Steps
a2c01	0.0007	0.99	5	0	0.5	1	+1.54	0.00	150.0
a2c02	0.001	0.99	8	0.01	0.5	0.95	+17.61	0.47	96.1
a2c03	0.0003	0.95	5	0	0.25	0.9	+0.52	0.00	150.0
a2c04	0.0005	0.99	16	0.02	0.5	0.95	+23.83	0.67	73.7
a2c05	0.001	0.999	32	0.01	0.5	0.98	+24.87	0.83	56.9
a2c06	0.0007	0.98	5	0.05	0.5	1	+30.37	0.87	54.0
a2c07	0.0002	0.99	8	0	0.75	0.92	+1.35	0.00	150.0
a2c08	0.001	0.97	16	0.01	0.5	0.95	+8.72	0.23	124.4
a2c09	0.0004	0.99	64	0	0.5	0.95	-29.99	0.00	150.0
a2c10	0.0007	0.99	5	0.03	0.3	0.9	+29.22	0.80	59.3

A2C gives the sharpest result in the whole study. Every single configuration with `ent_coef` set to 0.0, that is `a2c01`, `a2c03`, `a2c07`, `a2c09`, scores exactly 0.00 and runs the full 150 steps every episode. Every configuration with an entropy coefficient between 0.01 and 0.05 checks in on 0.23 to 0.87 of episodes. Without an explicit entropy bonus A2C commits to a poor action distribution early, and because it updates on rollouts as short as five steps it never gathers the evidence to escape. This is the one knob that separates a working A2C from a useless one on this task.

Results Discussion

The table below summarises the best run of each method before the individual figures are discussed. The spread between the best and the worst configuration is included because it is the clearest evidence that the environment is actually sensitive to tuning.

Method	Best config	Best mean return	Worst config return	Check-in rate	Biometric	Manual	Steps
DQN	dqn07	+34.65	+8.47	0.97	0.97	0.00	42.7
PPO	ppo10	+36.97	-5.96	1.00	1.00	0.00	38.9
A2C	a2c06	+30.37	-29.99	0.87	0.87	0.00	54.0

Method	Best config	Best mean return	Worst config return	Check-in rate	Biometric	Manual	Steps
REINFORCE	reinforce10	+37.23	-9.02	1.00	1.00	0.00	35.8

Cumulative Rewards

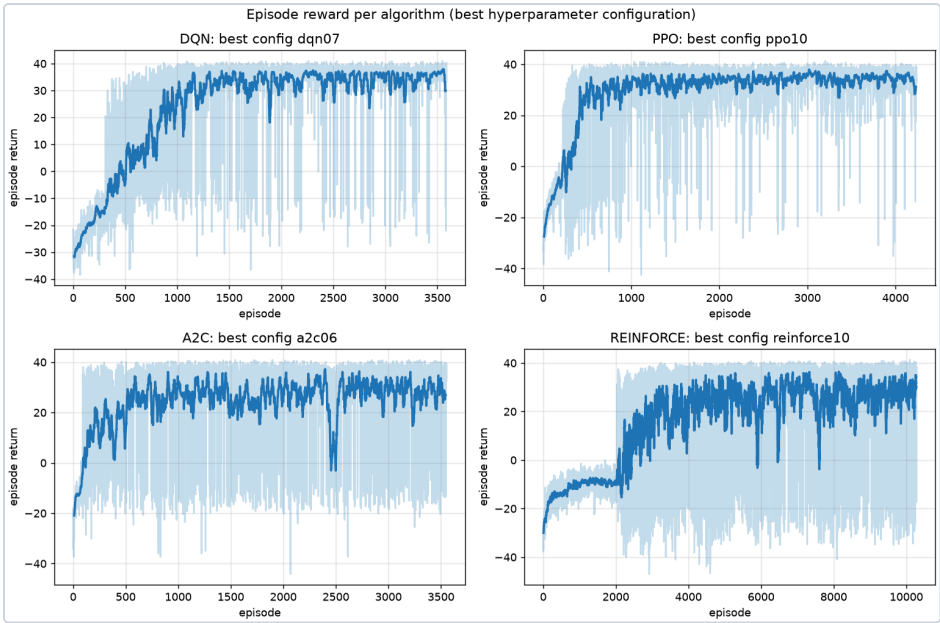


Figure 2. Episode return over training for the best configuration of each method. The pale line is the raw per episode return and the solid line is a 20 episode moving average.

The four curves have visibly different shapes. PPO rises fastest and then holds a tight band around +34, with only occasional single episode dips where a run of rejections or a broken reader B costs it the bonus. DQN climbs more slowly, takes until roughly episode 1200 to reach its plateau, and keeps showing deep downward spikes for the rest of training. Those spikes are the signature of a value based method with a moving target: a target network refresh can briefly change the greedy action in part of the state space. A2C reaches a reasonable level earliest of all but never becomes tidy, and its band stays wide from start to finish, which is what you expect from very short rollouts and single update steps. REINFORCE is the outlier. It sits flat near -10 for about 2000 episodes, doing nothing that looks like progress, then breaks through and climbs for the remaining 8000 episodes. That long flat spell is the exploration problem in its rawest form: until a Monte Carlo rollout happens to complete the whole sanitize then scan sequence, there is no gradient pointing towards it.

Training Stability

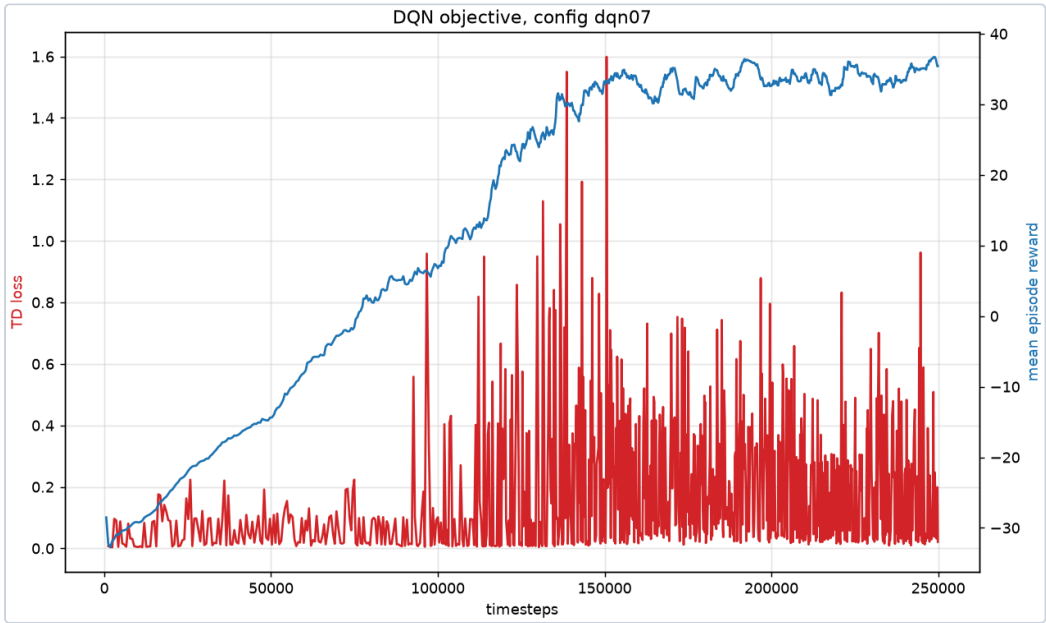


Figure 3. DQN objective for the best configuration. The red line is the temporal difference loss on the left axis and the blue line is mean episode reward on the right axis.

The DQN objective does not fall smoothly, and that is the correct behaviour rather than a fault. The temporal difference loss rises while mean reward is still climbing, because the agent is discovering states with much larger returns than its value estimates predicted, so the regression target keeps moving upward. The loss only settles once the reward curve flattens. Reading the two axes

together is what makes this interpretable: a falling loss on its own would be equally consistent with a policy that has stopped improving.

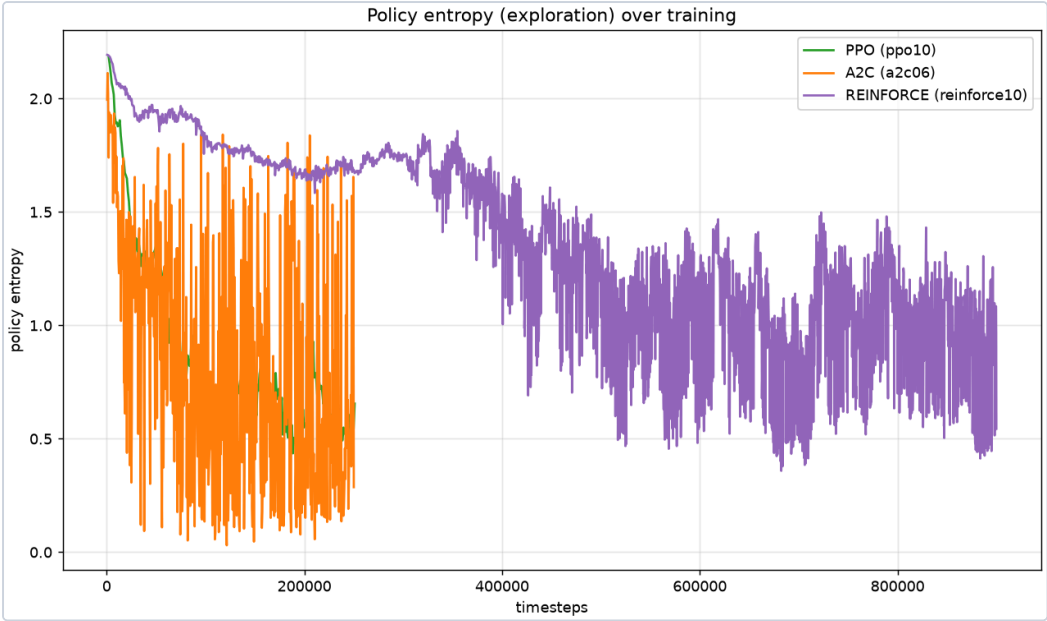


Figure 4. Policy entropy over training for the three policy gradient methods. Higher entropy means the policy is still spreading probability across actions.

The entropy curves explain the exploration story behind the sweep tables. All three start near the entropy of a uniform distribution over nine actions, which is about 2.2 nats. PPO and A2C both decay steadily as they commit to the route, and the entropy coefficient sets the floor they settle at, which is exactly why the A2C runs with no bonus collapse: their entropy falls away before they have found anything worth committing to. REINFORCE holds much higher entropy for far longer, which is the same fact as its long flat reward curve seen from the other side. It keeps sampling widely because its gradient estimate is too noisy to justify committing, and that is what eventually lets it stumble onto the full sequence.

Episodes To Converge

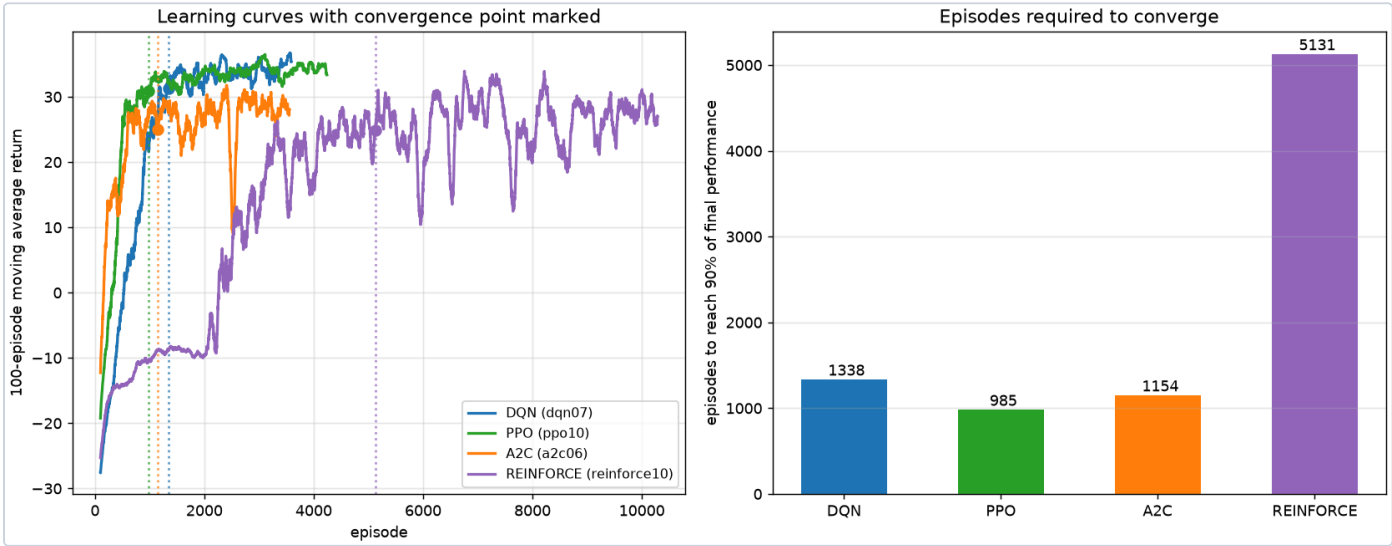


Figure 5. Left: 100 episode moving average for each method with its convergence point marked. Right: episodes needed to reach and hold 90% of final performance.

To put a number on convergence I take the plateau to be the mean return over the final 10% of episodes, then find the first episode whose 100 episode moving average reaches 90% of that plateau and holds it for the next 250 episodes. The holding requirement matters: without it a single lucky spike counts as convergence, and with a stricter rule that demands the average never drop again, the noisy methods only qualify in their last few hundred episodes.

Method	Best config	Episodes trained	First reached 90%	Held 90% (converged)	Final plateau return
DQN	dqn07	3590	1169	1338	+34.79
PPO	ppo10	4242	809	985	+34.33
A2C	a2c06	3555	597	1154	+27.78
REINFORCE	reinforce10	10288	3286	5131	+27.71

PPO converges first at episode 985, and it is also the only method whose first crossing at 809 is close to its stable point, a gap of only 176 episodes. A2C actually touches the 90% line earliest of all at episode 597, but then takes until 1154 to hold it, a gap of 557 episodes, which quantifies the instability visible in its reward curve. DQN needs 1338 episodes but reaches the highest plateau at +34.79. REINFORCE needs 5131 episodes, roughly 5.2 times PPO, and in sample terms the gap is wider still: PPO reached its plateau inside a 250k step budget while REINFORCE needed 900k. That ratio is the clearest statement of the cost of a pure Monte Carlo gradient on a task with a long action sequence.

Generalization

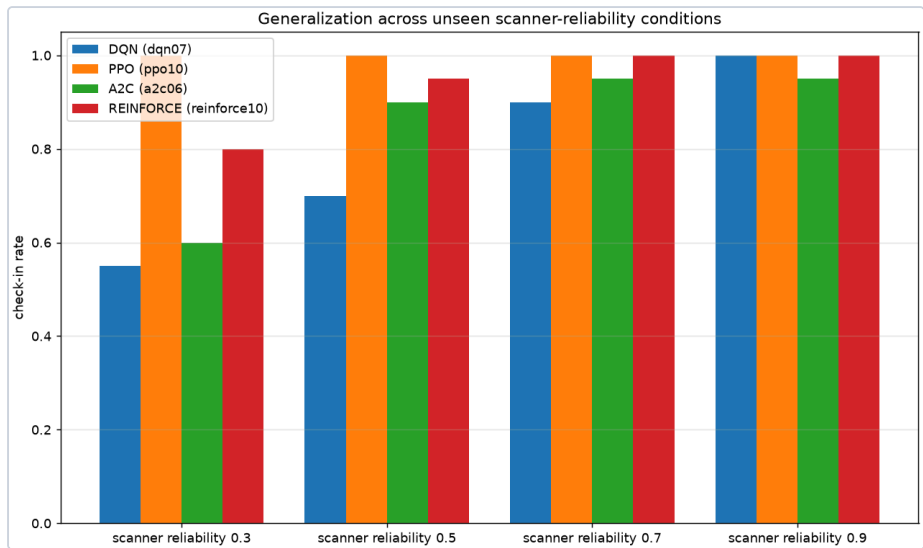


Figure 6. Check-in rate of each best agent when reader reliability is forced to values outside the training range. Training samples 0.80 to 1.00 for reader A and 0.45 to 0.85 for reader B.

This test holds the policies fixed and forces reader reliability to values they never saw, down to 0.3 where roughly two thirds of well presented fingers are rejected. PPO is completely unaffected and holds a 1.00 check-in rate at every level, including 0.3. That is not luck: with three attempts allowed per reader, two readers, and a staff call available, a policy that keeps trying can still get in through a bad reader, and PPO has learned to persist. A2C degrades gently, from 0.95 down to 0.60. DQN falls hardest, from 1.00 at reliability 0.9 to 0.55 at 0.3, which says its greedy policy is tuned to the reliability range it trained on and gives up rather than retrying when rejections mount. This is a useful reminder that the ranking on the training distribution, where DQN beat A2C comfortably, does not survive a shift in conditions.

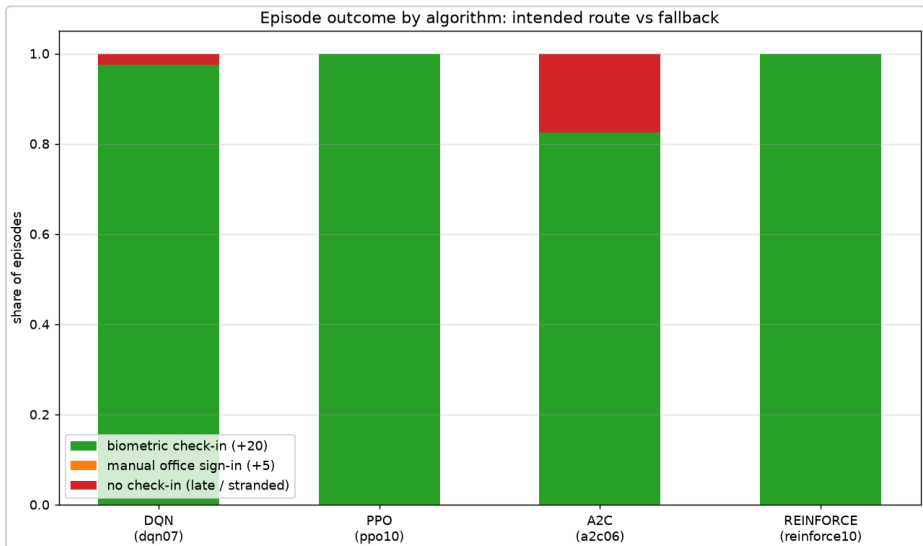


Figure 7. How each best agent ends its episodes: the intended biometric check-in, the lower value signature at reception, or no check-in at all.

This figure checks that the agents are solving the intended problem rather than taking the easy way out. Not one of the four ever signs the register, so the orange band is empty everywhere. Every success is a real fingerprint check-in. PPO and REINFORCE check in on every episode, DQN misses 2.5% and A2C misses 17.5%. Given that reception is a guaranteed +5 and is much closer to the gate than either reader, an agent settling for it would have been an entirely rational failure mode, and gating it behind a genuine lockout is what prevented that.

Agent Behaviour In The Simulation



Figure 8. Presenting a finger at reader A. The lamp is amber and the doors are shut.



Figure 9. The scan is accepted. The lamp turns green, the doors slide open and the pupil walks into the lobby.

Running `uv run main.py play` loads the best agent and shows it in the PyBullet window with a step by step trace in the terminal. The learned policy is unmistakably the intended one. In a typical episode the pupil starts with cleanliness 0.17, walks 13 steps north up the west corridor while its hands get dirtier still, takes three doses at the sanitizer to reach 1.00, crosses 15 steps east to reader A arriving at 0.82, is rejected once, and is accepted on the second attempt for a return of +40.15 in 35 steps, comfortably before the bell. In another episode it presented a finger twice into a busy reader, absorbed the small busy penalty, waited for the queue to drain and got in on the third try. That is the behaviour the reward was designed to produce: it takes the detour instead of gambling on dirty hands, and it persists at a contested reader instead of walking away.

Deployment and Frontend Integration

The environment is not only a training harness. The whole point of modelling a real school process rather than a grid world is that the same code can sit behind a product, so the environment and the trained agent are exposed as a JSON service in `api/serve.py`, built on FastAPI. A web or mobile client can render the courtyard and drive the agent over HTTP without importing a line of Python.

The JSON Contract

`api/schema.py` holds that contract in one place: the observation layout with the index range of every feature group, the nine actions, every reward constant, the dynamics and the scene geometry. `GET /manifest` serves it. The observation and action tables earlier in this report are generated from that same module, so the document you are reading and the contract a client consumes cannot describe different environments, and `tests/test_api.py` fails if the schema stops covering all 29 features.

```
GET /manifest      (abridged)
{
  "schema_version": "1.0.0",
  "env_id": "SchoolCheckIn-v0",
  "observation_space": {
    "type": "Box",
    "shape": [29],
    "dtype": "float32",
    "features": [
      {
        "index": [0, 2],
        "key": "position",
        "encoding": "float32, 2 values, scaled",
        "range": "[-1, 1]"
      },
      "... 13 more feature groups, 29 values in total"
    ]
  },
  "action_space": {
    "type": "Discrete",
    "n": 9,
    "actions": [
      {
        "id": 5,
        "name": "scan_fingerprint",
        "description": "Present a finger to a reader. Only works at a reader."
      },
      "... 8 more actions"
    ]
  },
  "rewards": {
```

```

    "biometric_check_in": 20.0,
    "manual_check_in": 5.0,
    "scan_rejected": -0.5,
    "step_cost": -0.05
  },
  "layout": {
    "scanners": [[9.0, 9.0], [9.0, 2.0]],
    "hygiene_station": [1.5, 8.0],
    "station_radius": 0.9
  }
}

```

Method	Path	Returns
GET	/manifest	this document
GET	/layout	scene geometry only
POST	/session	session_id + initial state
GET	/session/{id}	current state
POST	/session/{id}/step	state after the action
POST	/session/{id}/act	state after the trained agent picks one action
DELETE	/session/{id}	ends the session

A step is an ordinary POST. `/session/{id}/step` applies an action the client picked, and `/session/{id}/act` hands the decision to the best trained agent, so one session can be driven by a person, by the policy, or by both in turn. Every response carries the reward, the new state and the terminal flags, and it also carries the raw 29 value observation, which is exactly the vector the four agents were trained on, so a client is free to run its own model on it:

```

POST /session/{id}/step      {"action": 0}
{
  "action": "move_north",
  "action_id": 0,
  "state": {
    "position": [1.0, 1.5],
    "cleanliness": 0.163,
    "objective": [1.5, 8.0],
    "attempts_per_scanner": [0, 0],
    "locked": [false, false],
    "queue": [1, 0],
    "scanner_b_broken": null,
    "step": 1,
    "steps_to_bell": 79,
    "reward": 0.561,
    "terminated": false,
    "observation": [-0.8, -0.7, 0.8, 0.75, 0.775, 0.8, "... 23 more"]
  }
}

```

Two details in that payload are worth pointing out. `scanner_b_broken` is `null` rather than `false`, because the pupil has not yet been close enough to read the display, so the partial observability of the environment survives serialization instead of being quietly resolved by the API. And `objective` is the waypoint the reward shaping is currently measuring against, which gives a frontend something meaningful to draw without having to reimplement the potential function.

The Web Client

`web/index.html` is a single file with no framework, no build step and no dependencies: a canvas and `fetch`. The service mounts it at `/app`, so `uv run main.py serve` brings up the API and a working client together. It runs in two modes. Offline it replays `web/episode.json`, a full episode recorded frame by frame by `uv run main.py export` from the best trained agent, which means the demo works on a phone or a static host with no Python behind it at all. Pressing connect switches it to the live service, where `act` steps the real policy and the action buttons let a person take over mid episode. Because both modes consume the same state object, there is one rendering path rather than two.

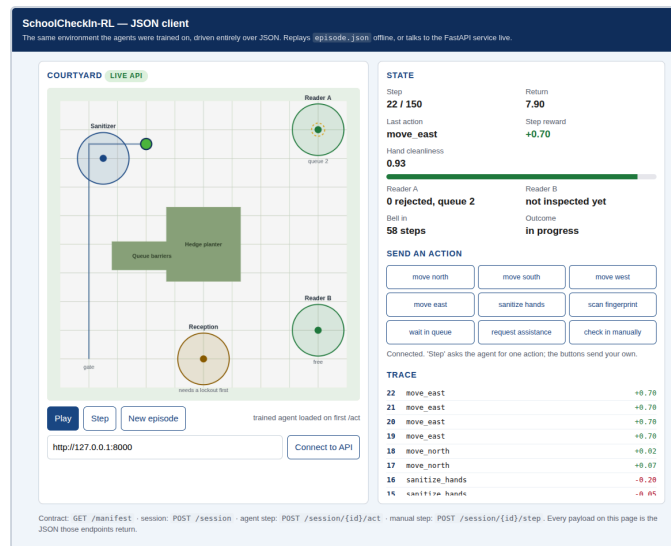


Figure 10. The bundled web client, connected to the live API. The courtyard, the readers with their queue and lockout state, the walked route and the hand cleanliness bar are all drawn from the JSON payloads shown above. Each button posts one action to the running service.

Running It On A Real Site

The service is stateless apart from a session dictionary, capped at 64 concurrent episodes and evicted oldest first; a real deployment would swap that for Redis and key a session to a pupil arriving at the gate. The policy is a frozen artifact loaded once on the first request, and inference is a single forward pass through two 64 unit layers, so the cost per decision is negligible next to the HTTP round trip and one small container comfortably covers a whole morning rush. Cross origin requests are allowed, so an Expo or React Native client and a browser client can share one backend, and the generated OpenAPI document at `/openapi.json` lets a mobile team produce typed models rather than hand written request code.

The observation table in the environment section is the integration contract for the rest of it. Every feature names the sensor, camera or API that would supply it on a real site, so connecting this to an actual gate means writing an adapter that fills the same 29 value vector from the attendance system, the device heartbeats and the queue camera, and nothing about the policy changes. The honest framing of the product is advisory rather than autonomous: the agent does not move a pupil, it recommends the next step, which a kiosk screen or a phone app can display as sanitize first, use the main door rather than the side gate, wait for the queue, or go to reception. The simulation is where that policy is trained, regression tested and compared against the previous one before it is ever shown to a child.

Conclusion and Discussion

On this environment the two strongest methods are PPO and REINFORCE with a learned baseline, which finish within 0.3 of each other at +37.23 and +36.97, both checking in on every evaluation episode. They get there very differently. PPO needed 250k steps and 985 episodes; REINFORCE needed 900k steps and 5131 episodes for the same result. If sample efficiency matters at all, and on a real gate every sample is a real pupil, PPO is the clear choice. DQN reaches the highest training plateau of any method at +34.79 and a 0.97 check-in rate, but it is the least robust: forced onto unreliable readers it drops to 0.55, the worst of the four. A2C is the weakest overall at +30.37 and is the most sensitive to a single hyperparameter, since every run without an entropy bonus fails completely.

The pattern behind those results is that this task punishes premature commitment. It needs a roughly 35 step sequence with a detour that looks like a detour, so any method that narrows its action distribution before finding the full sequence gets stuck. That is exactly what happens to A2C with no entropy bonus, to the DQN runs whose exploration schedule decays too fast, and to REINFORCE without a baseline. PPO does best because the clipped objective limits how far each update can move the policy, which keeps exploration alive without needing the entropy term tuned precisely.

The main strengths and weaknesses split cleanly. DQN reuses rare successful episodes many times through its replay buffer, which is why it reaches a high plateau, but its greedy policy transfers poorly and its value target keeps moving, which shows as permanent spikes in the reward curve. PPO is stable, sample efficient and by far the most robust off distribution, at the cost of more moving parts to configure. A2C is cheap per update and learns quickly at first, but its short rollouts make it noisy and fragile. REINFORCE is by far the simplest to implement and reasons about, and it does reach the top of the table, but it needed nearly four times the samples and it is unusable without the value baseline.

Two experiments shaped this work more than any tuning did, and both were reward design problems rather than algorithm problems. The first was the shaping term: rewarding progress toward whichever sub goal was currently active let PPO shuttle between the sanitizer and the reader for about +40 per episode at a 0% check-in rate. Rewriting it as a proper potential over remaining route length, so that closed loops sum to zero, removed the exploit. The second was the manual fallback: as an unconditional option it was a nine step walk for guaranteed reward and PPO took it every time, so it now requires a genuine lockout first. Both are recorded as regression tests, because in both cases the training curves looked healthy while the agent was not doing the task at all.

With more time the obvious next steps are a recurrent or frame stacked policy, since reader B's health is genuinely hidden and a memory of what was seen near the side gate would be worth more than the single masked feature the agent gets now; several random seeds per configuration, because a ten run sweep at one seed cannot separate a good setting from a lucky one; and multiple pupils sharing the readers, which would turn the queue from sampled noise into something the agent could reason about and would make the environment a much closer match to a real morning rush.

References

1. Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press. Section 13.4 for REINFORCE with a learned baseline.
2. Williams, R. J. (1992). Simple statistical gradient following algorithms for connectionist reinforcement learning. *Machine Learning* 8(3), 229 to 256.
3. Mnih, V. et al. (2015). Human level control through deep reinforcement learning. *Nature* 518, 529 to 533. Replay buffer and target network.
4. Mnih, V. et al. (2016). Asynchronous methods for deep reinforcement learning. *Proceedings of ICML*. The A3C paper that A2C is the synchronous form of.
5. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.
6. Schulman, J., Moritz, P., Levine, S., Jordan, M. and Abbeel, P. (2016). High dimensional continuous control using generalized advantage estimation. *Proceedings of ICLR*. arXiv:1506.02438.
7. Ng, A. Y., Harada, D. and Russell, S. (1999). Policy invariance under reward transformations: theory and application to reward shaping. *Proceedings of ICML*. The potential based shaping result this environment's reward relies on.
8. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. and Dormann, N. (2021). Stable-Baselines3: reliable reinforcement learning implementations. *Journal of Machine Learning Research* 22(268), 1 to 8.
9. Towers, M. et al. (2024). Gymnasium: a standard interface for reinforcement learning environments. arXiv:2407.17032.
10. Coumans, E. and Bai, Y. (2016 to 2021). PyBullet, a Python module for physics simulation for games, robotics and machine learning. <https://pybullet.org>
11. Paszke, A. et al. (2019). PyTorch: an imperative style, high performance deep learning library. *Advances in Neural Information Processing Systems* 32.
12. Ramirez, S. (2018 to 2025). FastAPI documentation. <https://fastapi.tiangolo.com>
13. Tabassi, E. et al. (2021). NIST Fingerprint Image Quality 2 (NFIQ 2). NISTIR 8382, National Institute of Standards and Technology. The image quality score cited as the real world source of the hand cleanliness feature.